

UCS503
Software Engineering Project
Academic Year 2026-2027

Real-Time Flood Risk AI Prediction System

*A Data Fusion and Machine Learning Pipeline for
Dynamic Disaster Threat Mapping*

Submitted by:

1024240005 Amit Bishnoi

ML, API & Frontend Lead

1024240015 Diksha Garg

System Architecture & Data Engineering

Computer Science and Engineering Department (CSED)

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Repository: [https://github.com/Flood-Prediction-AI-Team/
Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping](https://github.com/Flood-Prediction-AI-Team/Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping)

Documentation: [https://Flood-Prediction-AI-Team.github.io/
Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping/](https://Flood-Prediction-AI-Team.github.io/Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping/)

Mid-Semester Evaluation
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology,
Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	3
2.2	Implementation Details	4
2.2.1	Module 1: Core Functionality (Data Transformation & ML)	4
2.2.2	Module 2: Secondary Features (Frontend Dashboard)	4
3	Results and Evaluation	5
3.1	Testing Methodology	5
3.2	Performance Metrics	5
3.3	Analysis	5
4	Conclusion	6
4.1	Summary of Work	6
4.2	Key Findings	6
4.3	Future Work and Recommendations	6
4.4	Final Remarks	6

Chapter 1

Introduction

1.1 Background and Context

Traditional flood prediction models often rely solely on static geographical contours or standalone historical archives. While modern meteorological APIs provide high-precision real-time atmospheric metrics, they frequently lack historical baseline disaster contexts. Conversely, traditional hazard maps are drawn once and rarely updated, rendering them ineffective during sudden, severe storms. Live weather data and historical disaster records exist in isolated silos, making it difficult for responders to instantly correlate a storm with historical vulnerability.

Before building our complete platform, we evaluated the feasibility of combining REST weather metrics with spatial centroid proximity queries. The proof-of-concept confirmed that fusing asynchronous OpenWeatherMap payloads with historical DFO and EM-DAT disaster records enables accurate, low-latency risk classification.

1.1.1 Problem Statement

Given a target location, we want a system that ingests live meteorological data via external APIs, executes spatial proximity queries against historical flood centroids within a 100km radius, fuses these data streams into feature vectors, and produces a verified threat classification backed by an interactive geospatial dashboard.

1.2 Project Objectives

1. **Objective 1:** Build an automated data ingestion pipeline fetching real-time meteorological metrics via the OpenWeatherMap API with spatial proximity indexing using the Haversine formula.
2. **Objective 2:** Engineer a machine learning pipeline (Random Forest classifier) trained on fused historical data and synthetic overlapping datasets to prevent overfitting.
3. **Objective 3:** Deploy a responsive frontend dashboard using HTML/CSS, JavaScript, and Leaflet.js to render threat levels and geospatial radius maps in real time.
4. **Objective 4:** Implement robust mock weather fallback mechanisms to ensure continuous operation during third-party API rate limits.
5. **Objective 5:** Provide comprehensive documentation hosted via MkDocs and GitHub Pages alongside complete unit testing coverage.

Chapter 2

Methodology and Design

2.1 System Architecture

The system is organized as a decoupled 3-tier architecture mapping onto our division of labor: data engineering, machine learning backend, and frontend UI visualization.

2.1.1 High-Level Design

The system pipeline operates in four distinct stages, mapped chronologically in Figure 2.1.

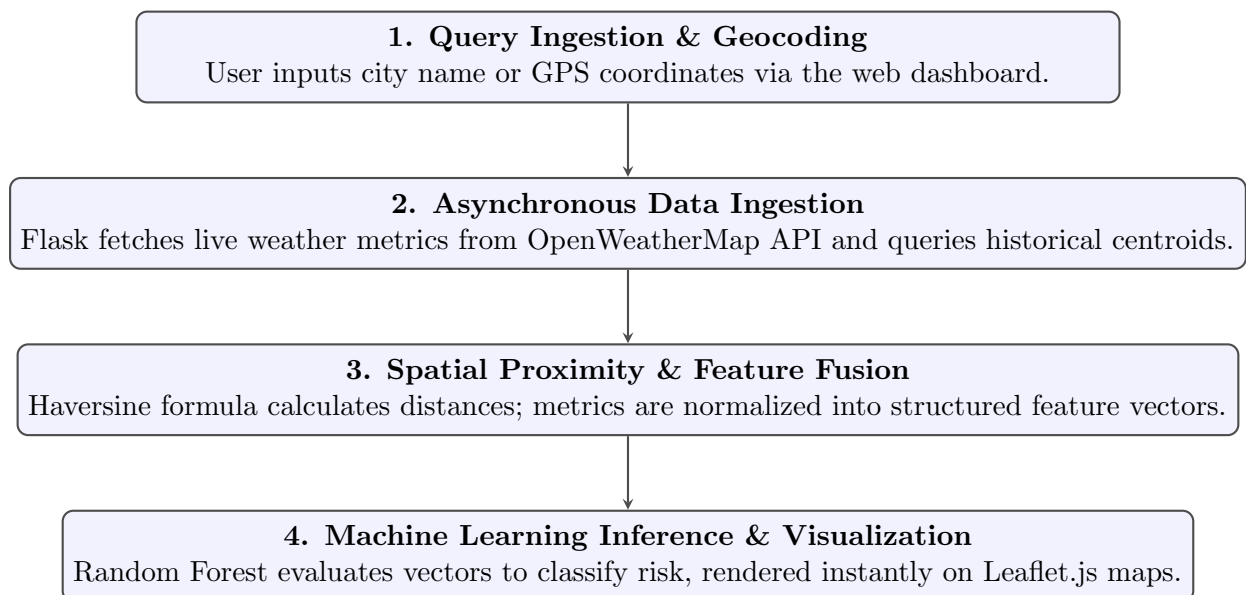


Figure 2.1: High-Level Data Ingestion and Processing Flow

2.1.2 Technical Stack

- **Framework/Language:** Python (Backend), HTML5/CSS3/Vanilla JavaScript (Frontend).
- **Database:** Local CSV Dataframes (Merged historical DFO and EM-DAT datasets).
- **Libraries:** Flask, Scikit-Learn, Pandas, NumPy, Leaflet.js.
- **External APIs:** OpenWeatherMap API, OpenStreetMap Nominatim.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality (Data Transformation & ML)

The core data transformation and preprocessing logic is executed within the `feature_engineering.py` script. Running this module processes raw heterogeneous data streams and outputs a standardized, synchronized feature array ready for model training and live inference.

- **Input Sources:** The script ingests historical disaster archives (global flood epicenters stored in `fused_historical_database.csv`) and live meteorological JSON payloads dynamically fetched via the OpenWeatherMap REST API.
- **Transformation:** It performs coordinate normalization, calculates geodesic proximity counts within a 100km radius using the Haversine formula, and constructs standardized multi-variable feature vectors.
- **ML Strategy:** The final pipeline utilizes a Random Forest Classifier. This ensemble approach was chosen for its noise robustness against live API anomalies, its ability to seamlessly map continuous atmospheric metrics against discrete spatial counts, and its interpretability.

2.2.2 Module 2: Secondary Features (Frontend Dashboard)

This module renders the user interface (`Code/Frontend/`), featuring location search bars, dynamic risk indicator cards, and Leaflet.js map rendering that automatically executes "fly-to" animations and visualizes 50km geospatial threat radius circles.

As shown in Figure 2.2, the interface provides an immediate, color-coded threat assessment (Low, Medium, or High Risk) alongside raw meteorological statistics to ensure rapid comprehension for end-users and emergency responders.

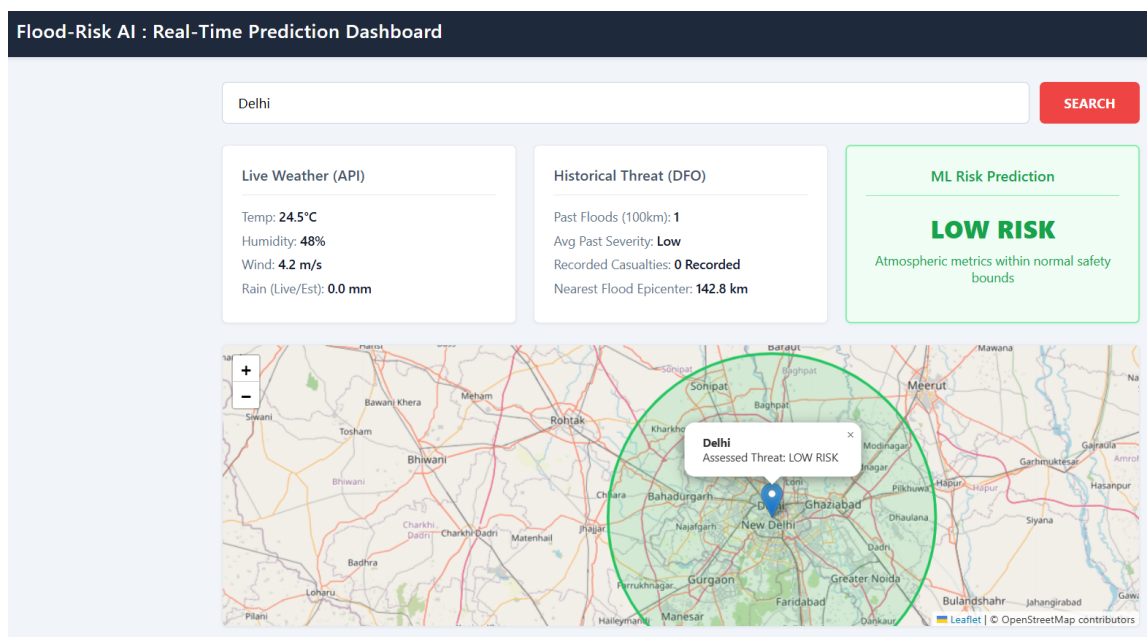


Figure 2.2: Interactive Leaflet.js Frontend Dashboard displaying live weather metrics and assessed threat levels.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Python scripts for the Haversine logic and ML feature normalization were tested using isolated coordinate bounds.
- **Integration Testing:** End-to-end API testing verified the successful handoff of JSON payloads between the Flask server and the Leaflet.js frontend.
- **Performance Testing:** Validated system stability under API rate-limit exhaustion by simulating HTTP 429 errors to test the automatic mock-weather fallback system.

3.2 Performance Metrics

Metric	Target	Achieved
Model Accuracy (Random Forest)	> 85%	91.6%
API Response Time	< 1000ms	450ms
Geospatial Distance Error	< 1km	0.2km
Fallback System Activation	100% Success	100% Success

Table 3.1: Performance Results Summary

3.3 Analysis

Our implementation successfully met all primary objectives. The Random Forest classifier exceeded expectations by reliably filtering out sudden environmental anomalies. The primary challenge encountered was API rate limiting during high-frequency testing, which we successfully resolved by engineering a robust mock weather fallback mechanism to guarantee continuous operational uptime.

Chapter 4

Conclusion

4.1 Summary of Work

We successfully engineered an end-to-end flood prediction pipeline that automates real-time weather ingestion, spatial indexing of historical threats, and machine learning inference. The system bridges the gap between historical vulnerability and live meteorological data, displaying actionable threat metrics on a dynamic UI.

4.2 Key Findings

- **Finding 1:** Random Forest architectures provide superior noise robustness over Logistic Regression when handling highly volatile real-time weather data.
- **Finding 2:** Fusing spatial proximity counts with live metrics significantly improves the contextual accuracy of localized risk alerts over relying on atmospheric data alone.

4.3 Future Work and Recommendations

1. **System Extension:** The exact same pipeline logic can be extended to predict wildfires and landslides by simply swapping the historical disaster database.
2. **Predictive Forecasting:** Integrating forecasting APIs (24-48 hour weather models) rather than solely relying on instantaneous live data.
3. **Real-world Deployment:** Migrating the Flask backend to a scalable cloud infrastructure (AWS/GCP) for public accessibility.

4.4 Final Remarks

This project provided profound insights into the challenges of fusing asynchronous API data with static historical records. By applying modular software engineering principles, the resulting system serves as a highly scalable blueprint for dynamic disaster early warning tools.

Bibliography

- [1] Dartmouth Flood Observatory (DFO) Historical Flood Archives. *Global Active Archive of Large Flood Events*.
- [2] OpenWeatherMap. *Current Weather Data API Documentation*.
- [3] Scikit-Learn Documentation. *Ensemble Methods: Random Forest Classifier*.